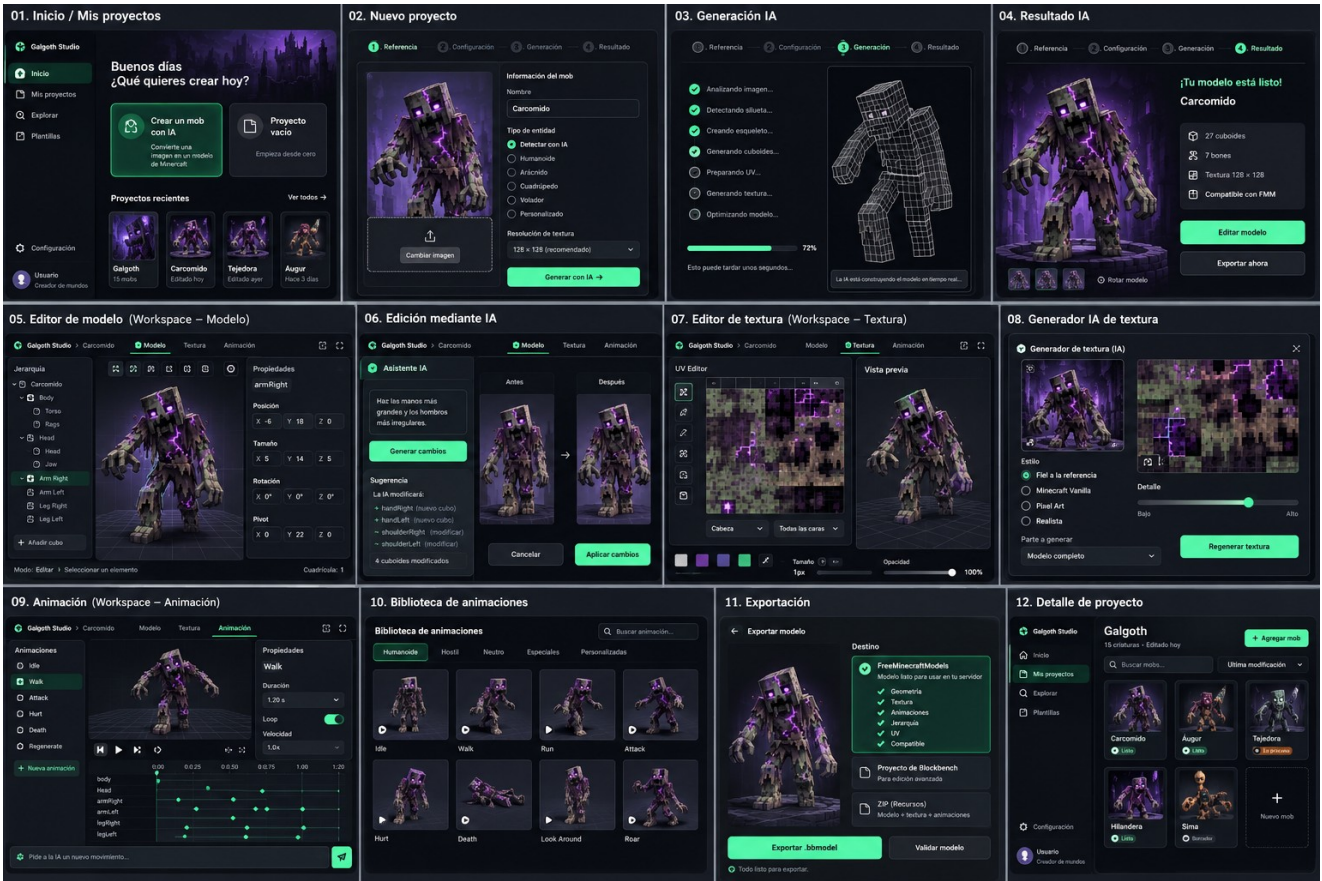


Galgoth Studio

Especificación de producto, UX, IA y arquitectura técnica

Editor web asistido por IA para crear modelos cuboidales de Minecraft Java, texturizarlos, animarlos y exportarlos a Blockbench / FreeMinecraftModels.



Mockup maestro: 12 vistas del flujo completo.

Documento build-ready

Incluye decisiones de producto, sistema visual, vistas, contratos de IA, editor 3D, UV/textura, animación, exportación .bbmodel, compatibilidad FMM, stack sugerido, roadmap y un prompt maestro para un agente de programación.

Versión: 2026-09-08

1. Resumen ejecutivo

Galgot Studio es una aplicación web enfocada en crear mobs y modelos para Minecraft Java Edition a partir de arte conceptual. La herramienta combina un editor 3D especializado en cuboides, un editor UV/pixel, un sistema de rigging y animación, y una capa de IA que genera y modifica propuestas estructuradas.

Principio central

La IA decide qué debería cambiar; el motor de la aplicación decide cómo representarlo, validarlo, versionarlo y exportarlo. Nunca se debe depender de que un LLM escriba directamente el .bbmodel final.



Pipeline completo: referencia → IA estructurada → motor determinista → exportación.

Objetivo del producto

- Convertir una imagen conceptual en una aproximación 3D Minecraft reconocible usando exclusivamente cuboides para el flujo principal.
- Permitir edición manual de bones, pivotes, cubos, UV, textura y keyframes.
- Hacer que usuarios sin experiencia en Blockbench puedan obtener un resultado válido mediante IA.
- Exportar un .bbmodel que pueda abrirse en Blockbench y consumirse por FreeMinecraftModels.
- Mantener proyectos con múltiples mobs y un flujo de trabajo no destructivo con historial y revisiones.

No objetivos del MVP

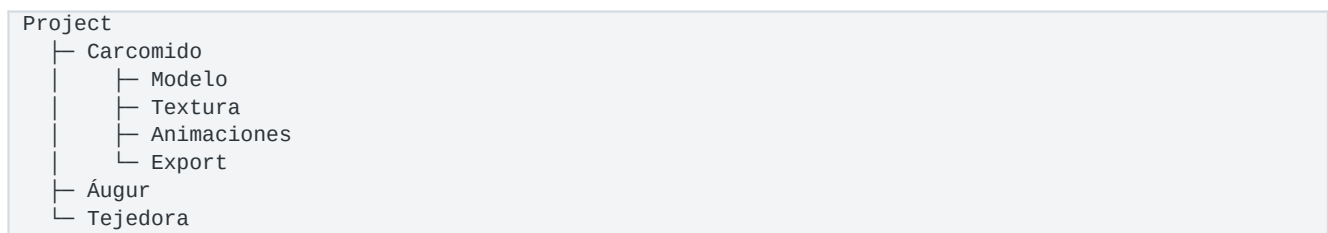
- No recrear Blockbench completo.
- No soportar Bedrock Edition en la primera versión.
- No admitir meshes arbitrarios como geometría principal del generador.
- No implementar un sistema Photoshop-style de capas de textura.
- No intentar inferir geometría físicamente perfecta desde una sola imagen; el objetivo es fidelidad visual Minecraft y buena animabilidad.

2. Decisiones de producto consolidadas

Área	Decisión actual
Plataforma	Web app desktop-first; gestión usable en tablet/móvil.
Edición objetivo	Minecraft Java Edition.
Proyecto	Un proyecto puede contener múltiples mobs.
Geometría	Cuboides + bones; pivotes preparados para animación.
IA	Propone cambios estructurados; la app valida y aplica.
Textura	Atlas UV editable y preview 3D en tiempo real; sin sistema general de capas.
Animación	Modo Simple con IA + modo Avanzado con timeline.
Exportación	.bbmodel + PNG/ZIP opcional; validación específica FMM.
Diseño	Dark graphite + acento verde menta; violeta reservado sobre todo para contenido de Galgoth.
Sidebar	Nuevo proyecto, Mis proyectos, Recientes; Configuración abajo. Sin “Volver”.

Proyecto vs. mob

La creación de proyecto debe seguir siendo simple. El wizard técnico de IA pertenece a la creación o agregado de un mob dentro del proyecto. Esto evita convertir “Nuevo proyecto” en un formulario complejo y mantiene la jerarquía: Proyecto → Mob → Modelo / Textura / Animación / Exportar.



Restricción importante sobre referencias visuales antiguas

No implementar capas

Algunos mockups antiguos muestran “Capas de textura / Overlay”. Esa exploración queda supersedida por la decisión actual: el MVP no tendrá un sistema general de capas. Las imágenes antiguas se incluyen solo por sus patrones de UV, preview 3D, proyecto y herramientas.

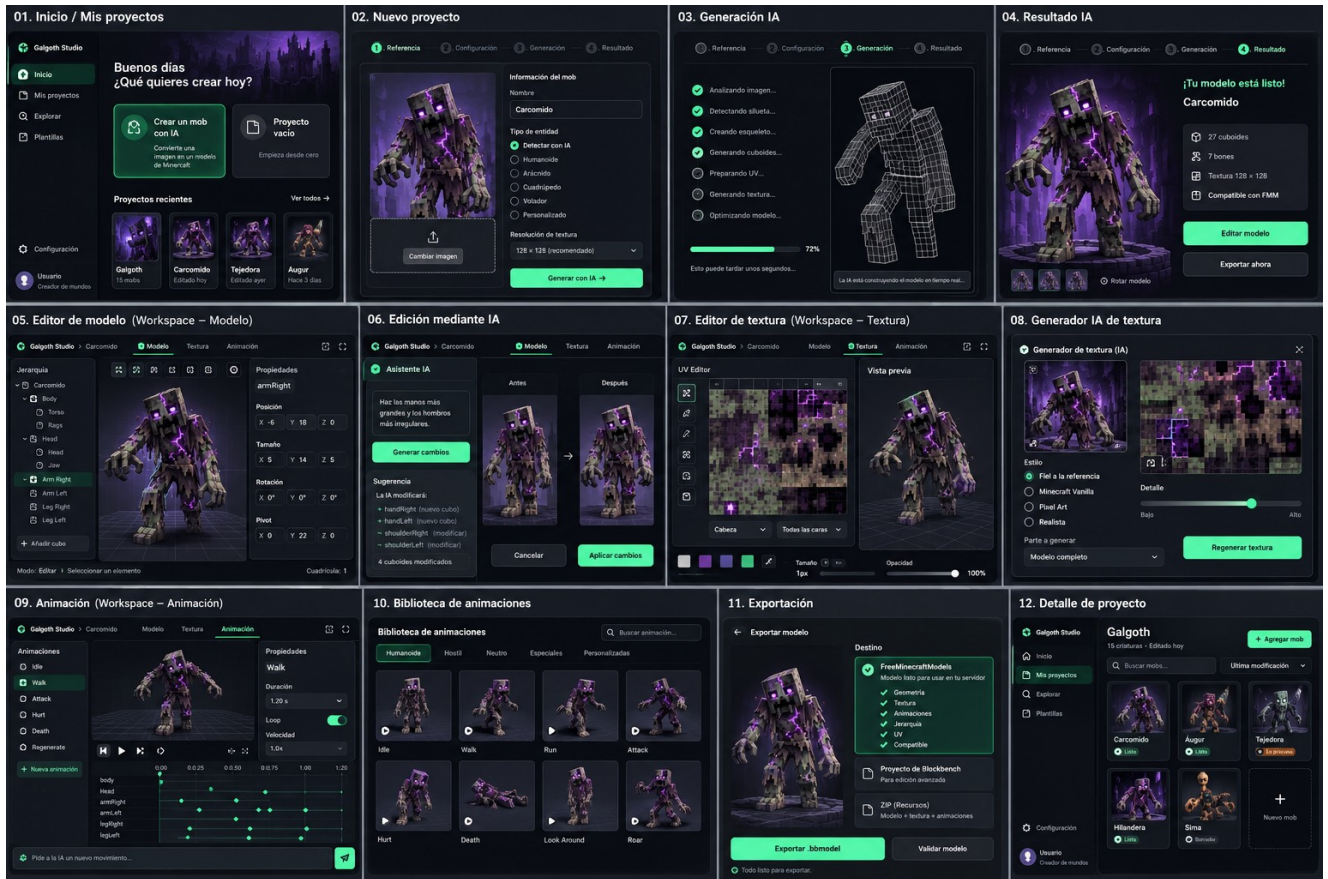
3. Sistema visual

La interfaz debe sentirse como una herramienta profesional creativa, no como un dashboard administrativo. El mob y su textura ocupan el espacio principal; los paneles se comportan como herramientas alrededor del contenido.

Token	Valor sugerido	Uso
--bg	#0B0F14	Fondo global
--panel	#111820	Paneles principales
--surface	#171F29	Controles y tarjetas
--border	#293540	Separadores 1px
--text	#F3F6F8	Texto principal
--muted	#93A0AA	Metadatos
--accent	#48E5A0	Selección/acción primaria/IA
--danger	#F46B6B	Errores y destructivas
--warning	#F2C66D	Advertencias

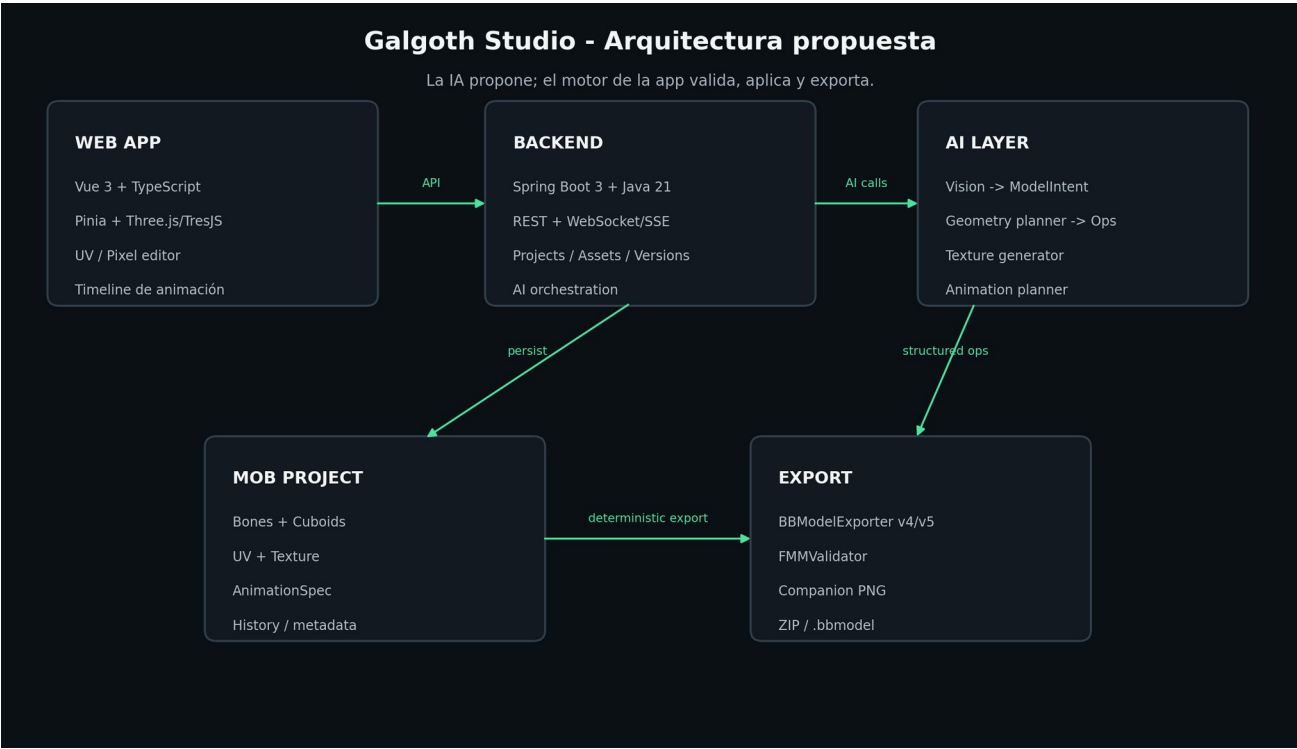
- Tipografía sugerida: Geist o Inter; JetBrains Mono / Liberation Mono para coordenadas, UV y keyframes.
- El verde menta se usa con moderación para selección, CTA, progreso y estados correctos.
- El violeta no debe invadir el chrome de la app: pertenece principalmente a criaturas Galgoth, magia y previews.
- Paneles oscuros, esquinas suavemente redondeadas, bordes sutiles y sombras contenidas.
- Editor con densidad profesional, pero acciones técnicas agrupadas y progresivas.

Referencia visual maestra



Las 12 vistas generadas en una sola lámina. Esta imagen tiene prioridad visual sobre exploraciones anteriores.

4. Arquitectura general



Arquitectura recomendada: cliente Vue, backend Java, capa de IA y exportador desacoplado.

Se recomienda un monolito modular inicialmente. El editor 3D y la interacción inmediata deben residir en el cliente; persistencia, IA, jobs y exportación pueden centralizarse en el backend.

Capa	Tecnología sugerida	Responsabilidad
Frontend	Vue 3 + TypeScript + Vite + Pinia	UI, editor 3D, UV/pixel, timeline, estado local.
3D	Three.js / TresJS selectivamente	Escena, cuboides, bones, gizmos, cámara.
2D	Canvas 2D / OffscreenCanvas	Atlas UV, pintura, selección y compositing.
Backend	Spring Boot 3.x + Java 21	Proyectos, AI orchestration, validación, exportación.
DB	PostgreSQL	Metadatos, revisiones, contratos estructurados.
Assets	S3-compatible	Referencias, texturas, thumbnails, exports.
Jobs	SSE o WebSocket	Progreso de IA y tareas largas.
Local	IndexedDB	Autosave/cache y resiliencia del editor.

Navegación y workspaces



La IA es contextual al workspace; Modelo, Textura y Animación comparten el mismo mob.

```

/projects
/projects/new
/projects/:projectId
/projects/:projectId/mobs/new
/projects/:projectId/mobs/:mobId/model
/projects/:projectId/mobs/:mobId/texture
/projects/:projectId/mobs/:mobId/animation
/projects/:projectId/mobs/:mobId/export
  
```

5. Vistas de la aplicación

Las siguientes 12 vistas forman el flujo visual principal. Los textos de los mockups son guía de composición; el producto real debe usar datos válidos y controles funcionales.

01. Inicio / Mis proyectos



Mockup — 01. Inicio / Mis proyectos

- Entrada principal a crear con IA y a proyectos recientes.
- Sidebar global mínimo: Nuevo proyecto, Mis proyectos, Recientes.
- Cards con previews reales; un proyecto puede enseñar máximo 3 miniaturas y +N.
- Evitar hero persistente por proyecto; la card debe representar el contenido real.

02. Nuevo proyecto / Configuración de mob

02. Nuevo proyecto

1. Referencia

2. Configuración

3. Generación

4. Resultado





Cambiar imagen

Información del mob

Nombre

Carcomido

Tipo de entidad

☒ Detectar con IA

☐ Humanoide

☐ Arácnido

☐ Cuadrúpedo

☐ Volador

☐ Personalizado

Resolución de textura

128 x 128 (recomendado) ▾

Generar con IA →

Mockup — 02. Nuevo proyecto / Configuración de mob

- La creación global de proyecto debe seguir simple; esta composición se reutiliza principalmente para “Agregar mob con IA”.
- Referencia a la izquierda; nombre, tipo base y resolución a la derecha.
- Detección automática del tipo con override manual.
- CTA único y claro: Generar con IA.

03. Generación IA



Mockup — 03. Generación IA

- Progreso por etapas, no spinner genérico.
- Mostrar wireframe/cuboides en construcción en cuanto sea posible.
- SSE/WebSocket para progreso; el job debe sobrevivir a reconexiones.
- Cancelar no debe destruir el estado previo del mob.

04. Resultado IA



Mockup — 04. Resultado IA

- Vista 3D interactiva con métricas: cuboides, bones, textura y compatibilidad.
- Acciones primarias: Editar modelo o Exportar ahora.
- No forzar al usuario a entrar al editor avanzado si el resultado ya le sirve.

05. Editor de modelo



Mockup — 05. Editor de modelo

- Jerarquía a la izquierda, viewport protagonista al centro, propiedades a la derecha.
- Herramientas: select/move/scale/rotate/pivot/add cube/add bone/duplicate/delete/undo/redo.
- El modelo debe estar en pose neutral; la postura dramática se reproduce en animación.
- Gizmos, snapping y selección clara.

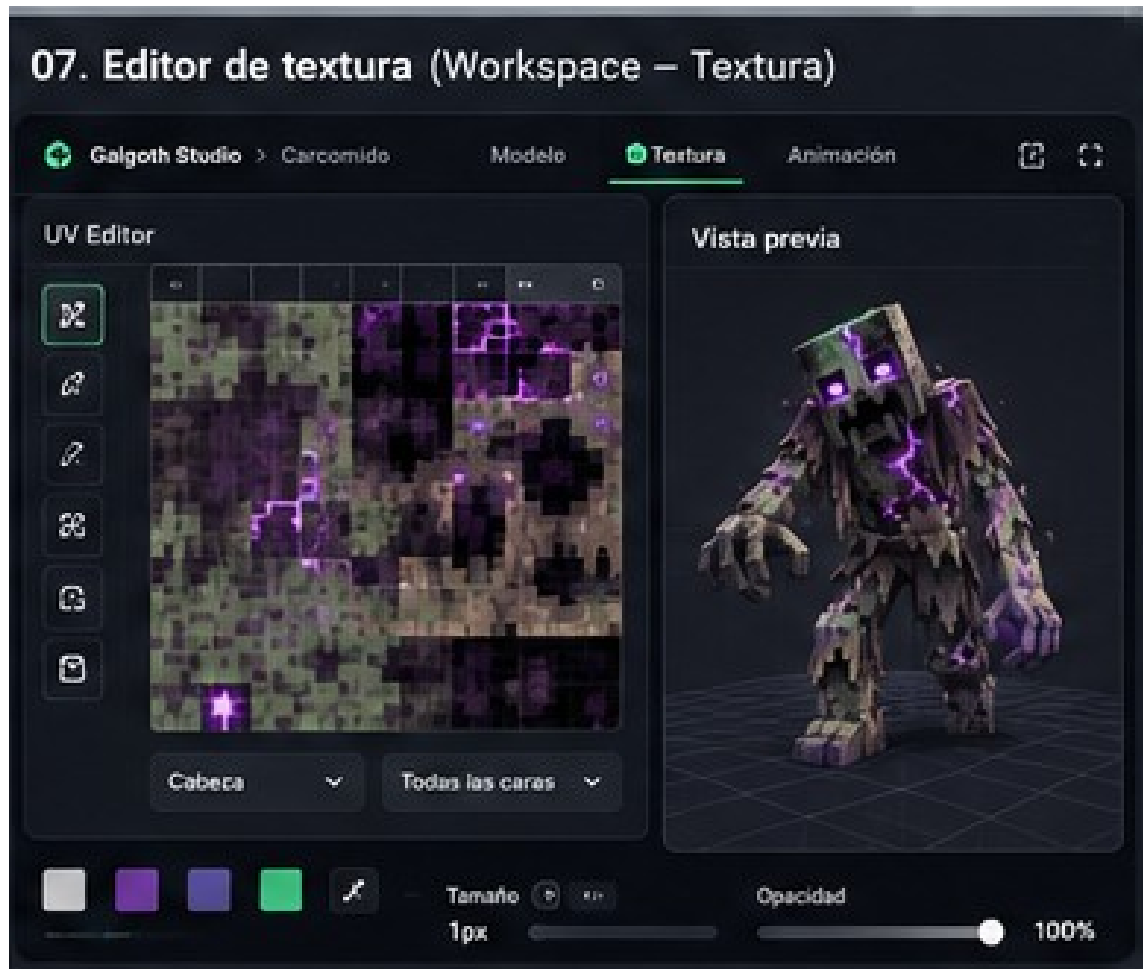
06. Edición mediante IA



Mockup — 06. Edición mediante IA

- Panel contextual, no chatbot separado.
- Mostrar Before / After y lista concreta de cuboides/bones modificados.
- Apply / Reject obligatorios antes de mutar el estado.
- AI edit = operaciones whitelist y schema-validación.

07. Editor de textura



Mockup — 07. Editor de textura

- Canvas UV editable + preview 3D en vivo.
- Guías UV por cara/parte; seleccionar en un lado resalta el otro.
- Herramientas: colores, pincel, borrador, cubeta, selector, copiar, pegar, recortar/ajustar, undo/redo.
- Soportar resoluciones variables por mob; sin sistema general de capas.

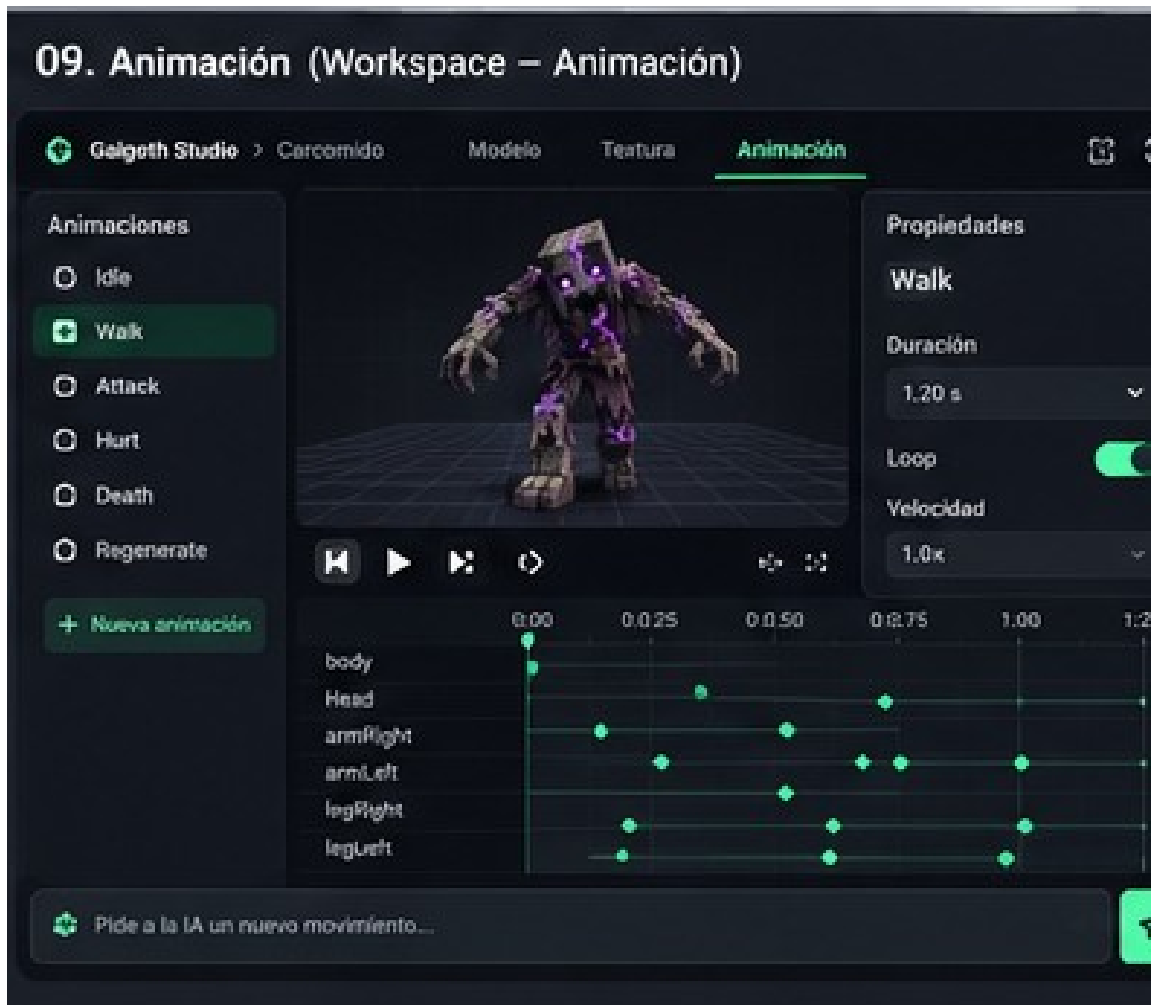
08. Generador IA de textura



Mockup — 08. Generador IA de textura

- Entrada conceptual + resultado sobre atlas/modelo.
- Estilos: fiel a referencia / Minecraft / pixel-art y control de detalle.
- Permitir regenerar solo cara, cabeza, torso u otra región.
- El compositor UV es determinista; el proveedor de imagen no controla directamente el archivo final.

09. Animación



Mockup — 09. Animación

- Lista de animaciones a la izquierda, viewport central, propiedades derecha y timeline inferior.
- Modo Simple para no animadores y modo Avanzado con keyframes.
- Playback 0.25x/0.5x/1x/2x, loop, scrubber y selección por bone.
- La IA edita tracks concretos cuando la instrucción es local.

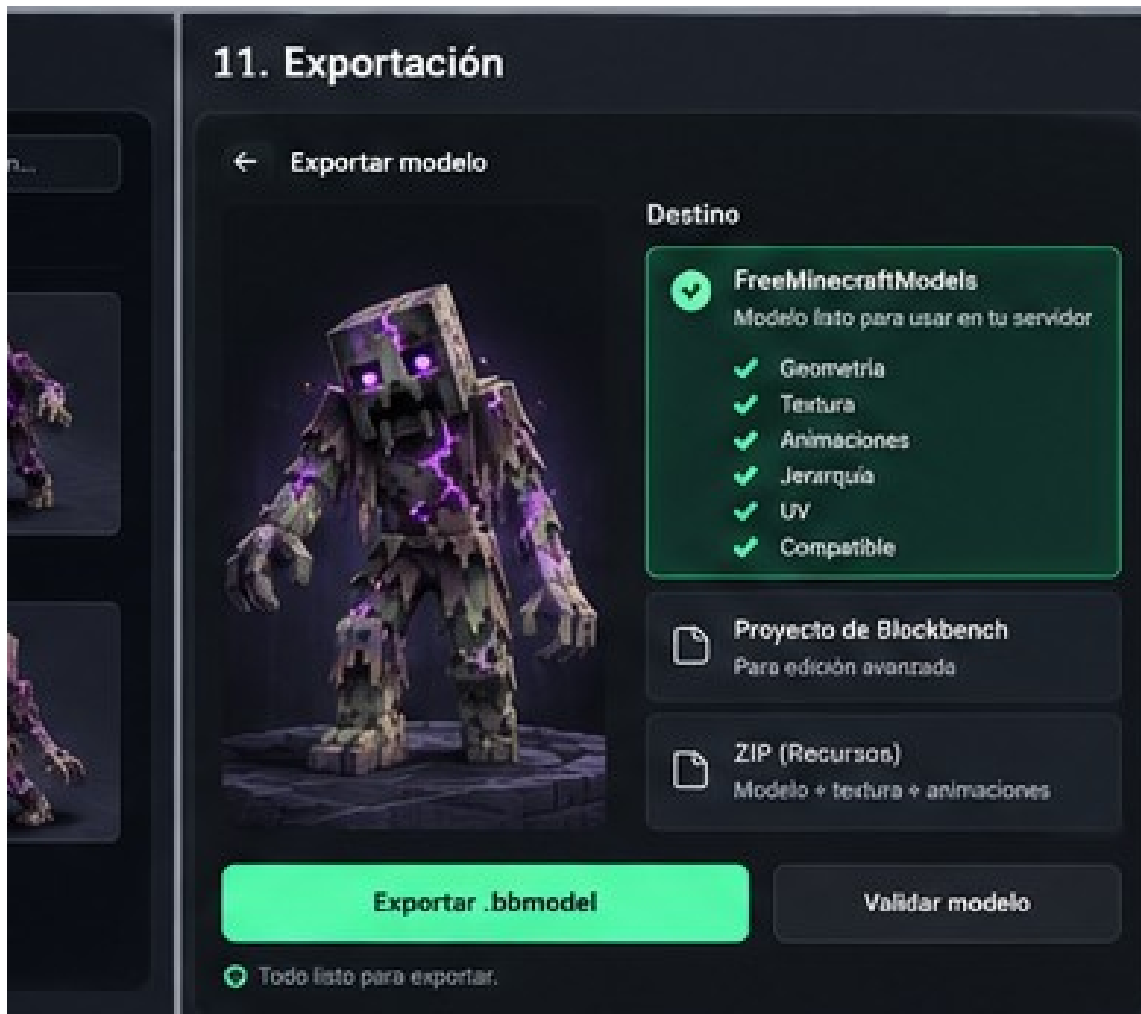
10. Biblioteca de animaciones



Mockup — 10. Biblioteca de animaciones

- Presets por familias: Humanoide, Hostil, Neutral, Especiales y Personalizadas.
- Cada card debe previsualizar el movimiento con el mob actual, no con una figura genérica.
- Usar preset + modificación IA para resultados consistentes.

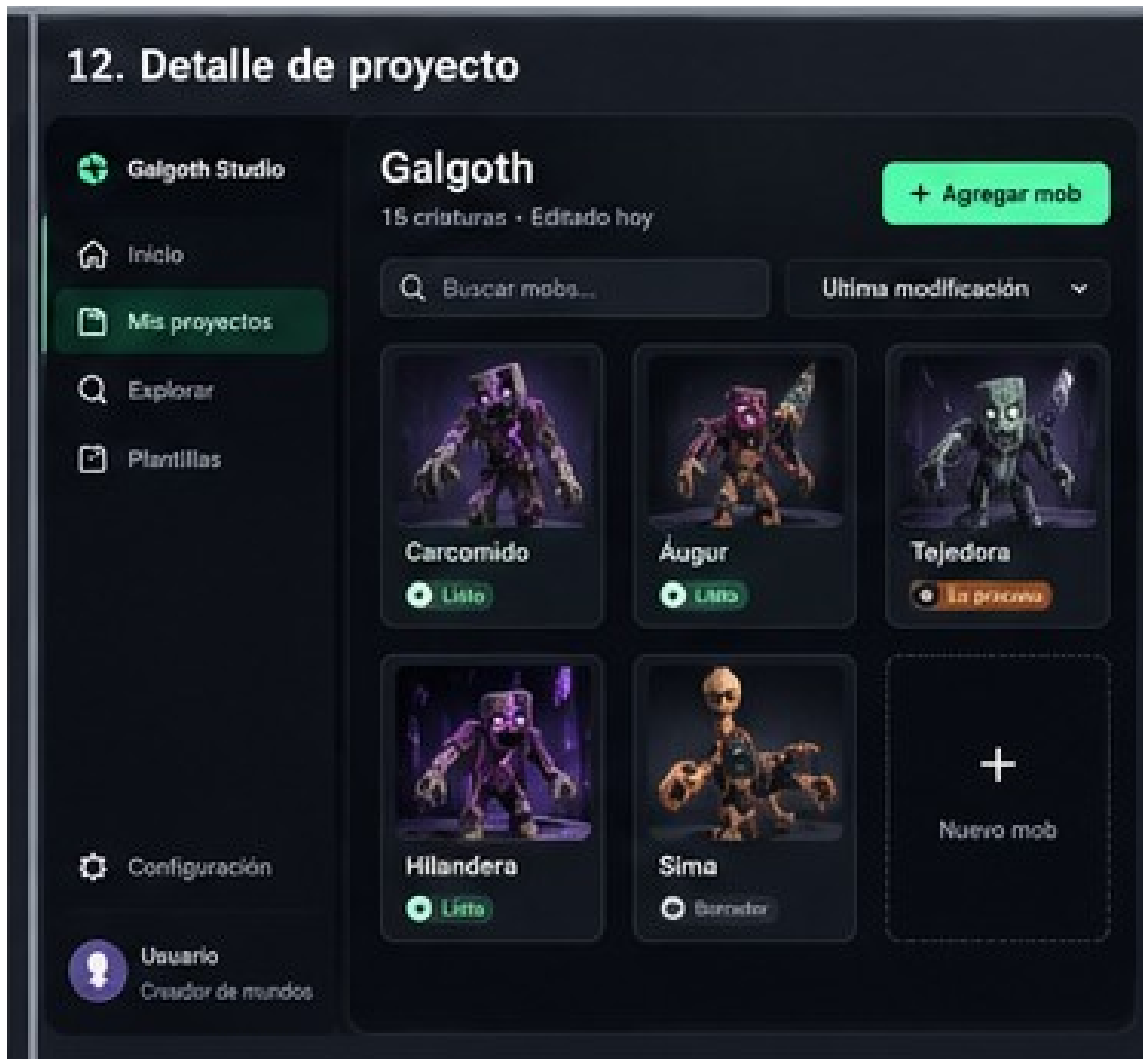
11. Exportación



Mockup — 11. Exportación

- Presentar compatibilidad como checklist entendible.
- FMM como destino principal y proyecto Blockbench como alternativa.
- Errores deben ser accionables; ofrecer auto-fix solo si la corrección es determinista.
- Exportar .bbmodel y opcionalmente ZIP con textura/companions.

12. Detalle de proyecto



Mockup — 12. Detalle de proyecto

- Colección de mobs con búsqueda, estado, previews y “Agregar mob”.
- Proyecto = unidad organizativa; mob = unidad editable.
- Las cards llevan a Modelo/Textura/Animación del mob.

6. Referencias visuales anteriores que siguen aportando valor

Estas exploraciones precedieron al storyboard de 12 vistas. No deben copiarse literalmente cuando contradigan decisiones actuales, pero contienen patrones útiles de proyecto, UV, modal “Agregar mob” y landing/branding.

Landing y lenguaje visual


Texture Studio
 Editor de texturas para Minecraft

Características
 [Galería](#)
[Cómo funciona](#)
[Preguntas](#)

[Descargar](#)

Da vida a tus ideas en cada píxel

Texture Studio es un editor de texturas para Minecraft Java Edition. Crea, modifica y organiza las texturas de tus mobs de forma sencilla, visual e intuitiva.

[Descargar gratis](#)
[Ver demo](#)

Para Minecraft Java
 Ligero y rápido
 Sin configuraciones complicadas

Editor visual
 Interfaz intuitiva con guías y previsualización 3D.

Todos los mobs
 Soporte para entidades de Minecraft Java Edition.

Organiza tus proyectos
 Gestiona múltiples mobs en un mismo proyecto.

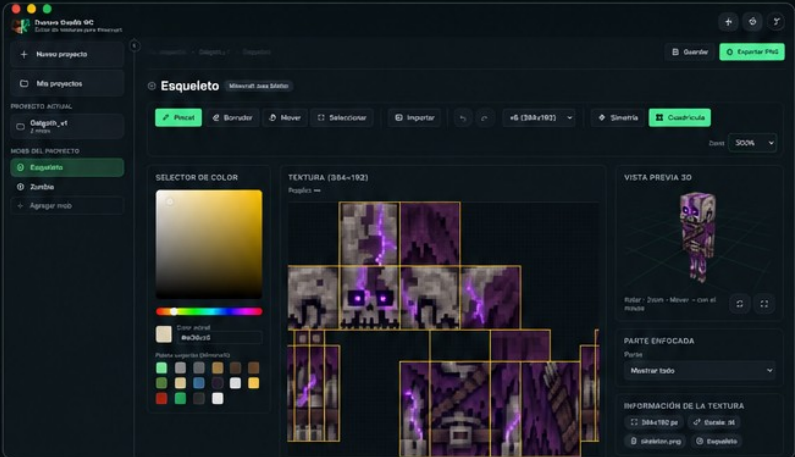
Exporta y comparte
 Genera tus texturas en PNG y llévalas a tu mundo.

CREA SIN LÍMITES

Un editor pensado para creativos

Pinta, prueba y visualiza tus texturas en tiempo real. Con herramientas simples y potentes, podrás enfocarte en lo importante: tu creatividad.

- Vista previa 3D interactiva
- Selector de color avanzado
- Herramientas esenciales
- Soporte para múltiples resoluciones
- Interfaz limpia y moderna

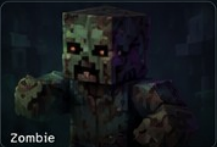


TU ESTILO, TU MUNDO

Texturas que cuentan historias

Desde criaturas sombrías hasta mundos completamente nuevos. La única limitación es tu imaginación.

[Ver galería](#)

 Esqueleto
  Zombie
  Araña
  Creeper
  Enderman

EMPIEZA HOY

Convierte tu visión en texturas reales

Únete a una comunidad de creadores y lleva tus ideas directamente a Minecraft.

[Descargar gratis](#)
[Ver en GitHub](#)

CREA
MODIFICA
COMPARTE

EL LÍMITE ES
TU IMAGINACIÓN

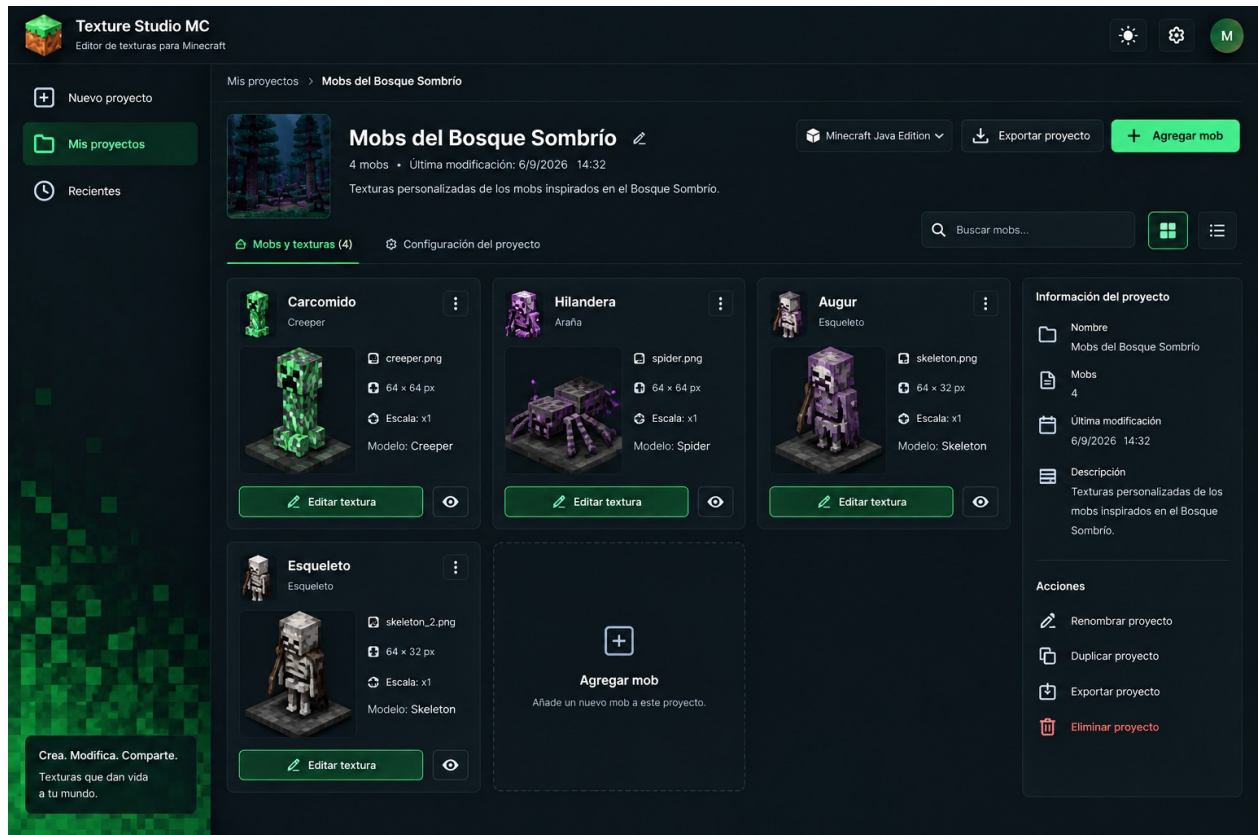

Texture Studio
 Editor de texturas para Minecraft

[GitHub](#)
[Discord](#)
[Contacto](#)




Útil para el lenguaje dark + mint, Minecraft/voxel y comunicación de producto.

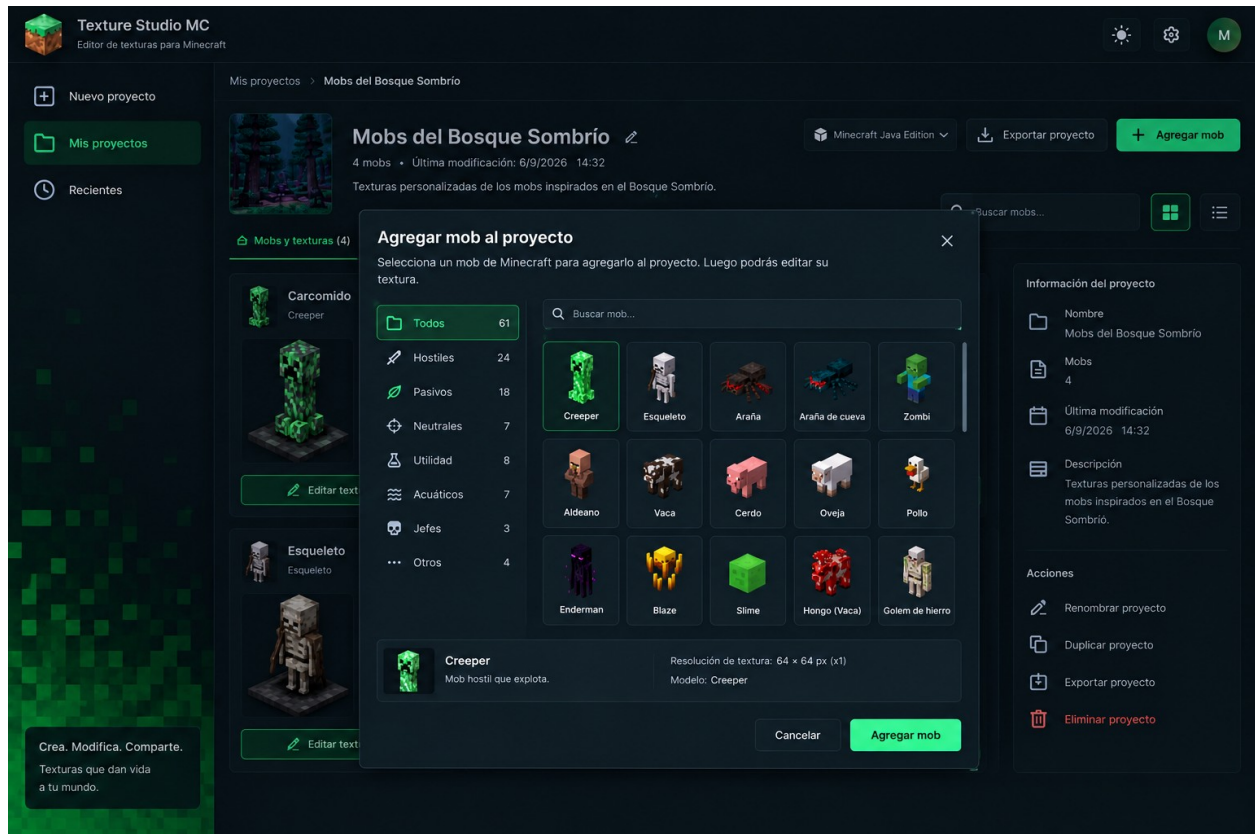
Vista de proyecto



Vista de proyecto

Útil para cards de mobs, búsqueda, acciones de proyecto y organización multi-mob.

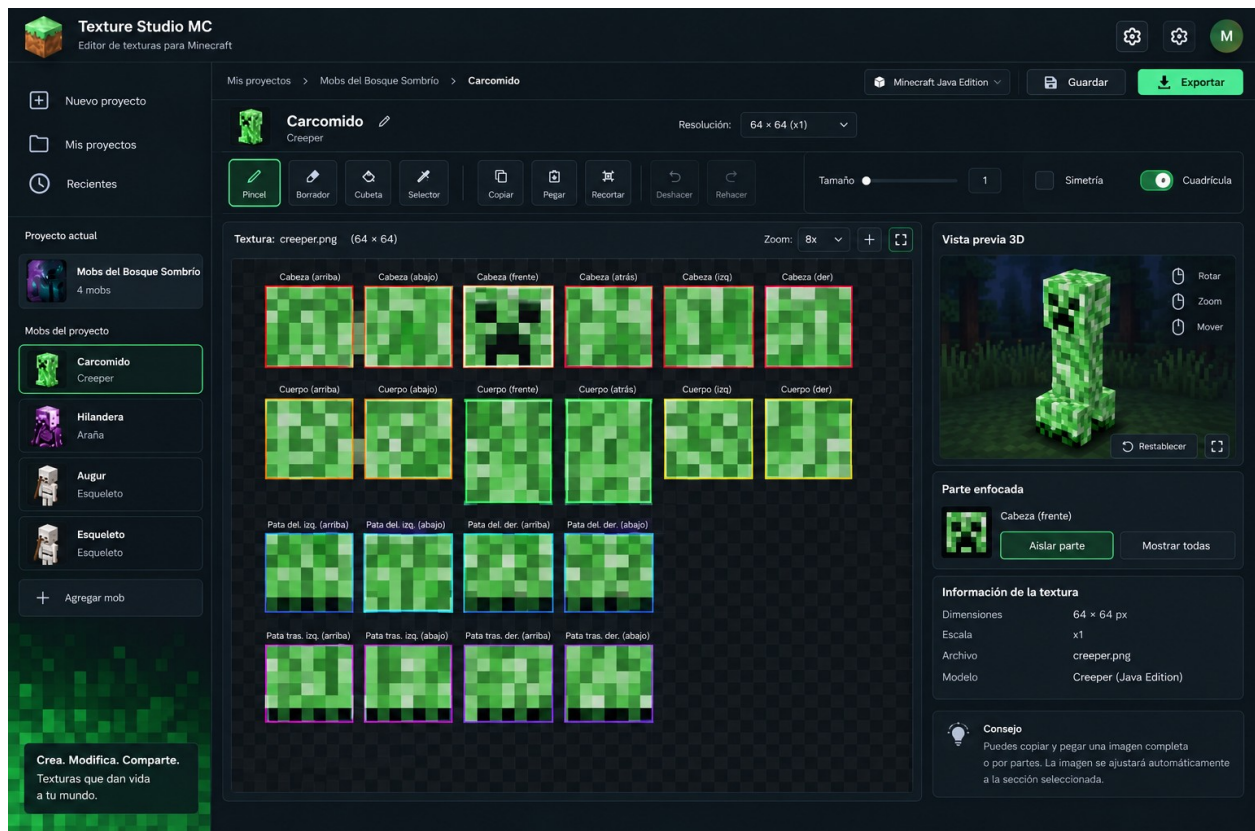
Agregar mob



Agregar mob

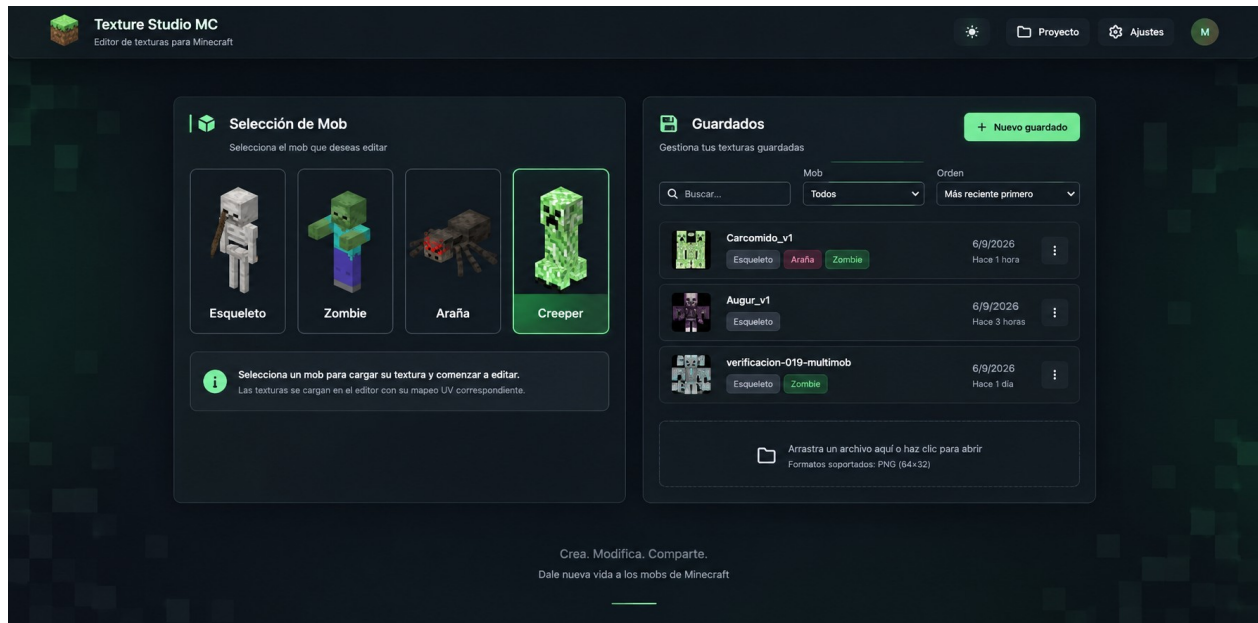
Útil como patrón modal para sumar un mob existente/vanilla o iniciar generación IA.

Editor UV anterior



Útil para guías UV, enfoque por parte y preview 3D. Ignorar cualquier referencia a capas.

Dashboard anterior



Dashboard anterior

Útil como referencia de jerarquía y selección, no como IA final.

7. Caso de prueba canónico: Carcomido



Referencia conceptual usada para probar imagen → modelo → textura → animación.

Carcomido debe ser la prueba de integración del producto porque contiene retos reales: silueta humanoide pero deformada, manos sobredimensionadas, asimetría, ropa irregular, brillo violeta, pose dramática y necesidad de separar geometría de animación.

Aspecto	Interpretación para el editor
Cabeza	Base cúbica; deformación moderada sin perder lectura Minecraft.
Manos	Más grandes que un zombie vanilla; geometría adicional si mejora silueta.
Brazos	Algo más largos/pesados; pivotes correctos para swing/ataque.
Torso	Humanoide, deteriorado; hombros irregulares con cuboides secundarios.
Ropa	Jirones mediante cuboides finos donde cambien la silueta.
Grietas violetas	Principalmente textura/emisivo visual, no geometría salvo relieve intencional.
Pose de referencia	No hornear en geometría; aproximarla con animación/pose de preview.

Fixture incluido

El paquete incluye samples/carcomido_minecraft_cuboids.bbmodel como fixture inicial. Tiene 25 cuboides y sirve para probar importación/exportación temprana, pero no debe convertirse en la estructura interna de la aplicación ni en el contrato definitivo del exporter.

8. Modelo de dominio interno

El proyecto persistido debe ser neutral respecto a Blockbench. Esto permite evolucionar el editor aunque cambie .bbmodel y habilita exportadores versionados.

```
interface MobProjectModel {
  id: string;
  projectId: string;
  name: string;
  baseType: 'humanoid' | 'arachnid' | 'quadruped' | 'flying' | 'custom';
  referenceImages: ReferenceImage[];
  bones: Bone[];
  cuboids: Cuboid[];
  texture: TextureDocument;
  uv: UvLayout;
  animations: AnimationSpec[];
  exportSettings: ExportSettings;
  revision: number;
}
```

Entidades principales

Entidad	Campos clave
Project	id, name, mobs[], createdAt, updatedAt
Mob	id, projectId, name, baseType, references[], currentRevision
Bone	id, name, parentId, pivot[3], rotation[3]
Cuboid	id, boneId, from[3], to[3], origin[3], rotation[3], faces
TextureDocument	width, height, bitmap asset, palette metadata
UvLayout	face regions, atlas coordinates, semantic labels
AnimationSpec	duration, loop, tracks, keyframes, events
Revision	number, actor, accepted operation batch, timestamp
ExportSettings	bbmodelVersion, embedTexture, fmm profile

9. Arquitectura de IA

Dividir la inteligencia en tareas pequeñas y con salida estructurada reduce errores y permite validar cada paso.

1. Vision analiza imagen y produce ModelIntent.
2. Geometry planner traduce intención a operaciones permitidas.
3. El motor aplica operaciones sobre el modelo interno y valida invariantes.
4. UV packer genera layout determinista.
5. Texture AI crea materiales/regiones; compositor construye el PNG final.
6. Rig validator revisa pivotes y jerarquía.
7. Animation AI parte de presets y devuelve AnimationSpec.
8. Exporter genera .bbmodel y FMMValidator emite reporte.

Contrato de operaciones geométricas

```
createBone(...)
createCuboid(...)
resizeCuboid(...)
moveCuboid(...)
rotateCuboid(...)
setBonePivot(...)
setBoneRotation(...)
parentBone(...)
removeCuboid(...)
```

Nunca aceptar UUIDs de la IA

IDs, UUIDs y referencias finales deben ser generados por el motor. La IA se refiere a IDs lógicos ya existentes o solicita nuevas entidades sin decidir sus identificadores persistentes.

Diff de IA

Cada operación IA crea una propuesta. La UI muestra resumen, cantidad de objetos afectados y preview antes/después. Solo al pulsar “Aplicar” se crea una nueva revisión. Un fallo de IA nunca debe alterar el estado actual.

10. Motor de geometría cuboidal

Regla	Validación
Cuboides solamente	No crear meshes en el flujo MVP.
Dimensiones	to[n] > from[n] para los tres ejes.
Jerarquía	Todos los parentId existen; no hay ciclos.
Pivotes	Los bones animables tienen pivotes coherentes.
UV	Cada cara usada apunta a una región válida.
Rotaciones	Se normalizan a los límites soportados por el formato/perfil objetivo.
Neutral pose	No hornear pose dramática del concept art como posición permanente.

Para humanoides usar una plantilla Minecraft como punto de partida, pero permitir escalas y cuboides secundarios para capturar identidad.

```
head  8 x 8 x 8
body  8 x 12 x 4
arms  4 x 12 x 4
legs  4 x 12 x 4
```

11. Textura y UV

La UV no debe ser improvisada por el modelo generativo. El editor mantiene regiones semánticas por cara/cuboide y un empaquetador determinista. La IA ayuda a construir el aspecto, no la topología del atlas.

- Selección bidireccional entre cara 3D y región UV.
- Guías coloreadas y etiquetas legibles cuando haya espacio.
- Focus por parte: resaltar una cara/parte y atenuar el resto.
- Pegar una imagen completa o por parte y ajustarla a la región seleccionada.
- Previsualización 3D sin animación obligatoria, con rotación y zoom.
- Resolución variable según el mob/proyecto; no asumir 64×64 para todos.

Pipeline de textura IA

```
Reference Image
-> palette/material analysis
-> semantic face plan
-> deterministic UV layout
-> per-region generation/inpainting
-> atlas compositor
-> pixel cleanup / resampling
-> preview + diff
```

12. Fase de animación

La animación entra después de estabilizar bones y pivotes. Internamente se representa con AnimationSpec y solo al exportar se adapta al formato de Blockbench.

```
AnimationSpec
  id
  name
  duration
  loop
  tracks[]
    boneId
    channel: rotation | position | scale
    interpolation
    keyframes[]
  events[]
```

Modo Simple

- Tipo: Idle, Walk, Run, Attack, Hurt, Death, Special.
- Duración y loop.
- Sliders de velocidad, peso, agresividad y asimetría.
- Prompt natural: “camina lentamente arrastrando la pierna derecha”.
- Preview inmediato y Regenerar.

Modo Avanzado

- Timeline con rows por bone.
- Keyframes editables y scrubber.
- Play/Pause, loop y velocidades 0.25x, 0.5x, 1x, 2x.
- Edición numérica en panel derecho.
- Eventos semánticos separados de transform tracks.

Estrategia IA para animación

La IA debe modificar presets, no reinventar locomoción básica innecesariamente. Por ejemplo: HumanoidWalkPreset + rig real + personalidad visual del Carcomido + prompt → AnimationSpec.

Carcomido / Walk

Preset humanoide pesado + inclinación de torso + swing asimétrico + arrastre de pierna derecha + cabeza inestable. Si el usuario pide “mueve menos el brazo izquierdo”, solo debe reescribirse el track necesario.

13. Exportación Blockbench y FreeMinecraftModels

La compatibilidad debe implementarse como adapters versionados. Blockbench documenta .bbmodel como formato interno sujeto a cambios; Blockbench 5.0 separó groups y outliner. FreeMinecraftModels actual detecta la versión mayor desde meta.format_version y maneja diferencias v4/v5.

```
MobProjectModel
├─ BBModelExporterV5 (preferido)
├─ BBModelExporterV4 (compatibilidad/fixture)
└─ FmmCompatibilityValidator
```

Validaciones mínimas

- JSON válido y meta.format_version explícito.
- UUIDs únicos.
- groups/outliner coherentes con el formato objetivo.
- Referencias de elementos/bones existentes.
- Dimensiones válidas.
- UV dentro del espacio de resolución.
- Índices de textura válidos.
- Keyframes apuntan a bones existentes y tiempos dentro de duración.
- Nombres especiales FMM válidos cuando se usan.

Convenciones FMM útiles

Convención	Uso
hitbox	Anchor/bone de colisión según FMM.
tag_name	Anchor/bone de nametag.
mount_<name>	Punto de montaje/asiento rideable.
Sibling .png/.yaml/.json	Archivos companion admitidos junto al modelo.

En la UI estas opciones deben vivir bajo una sección “FMM / Server” avanzada; no deben saturar el flujo creativo normal.

14. API y servicios

```
POST /api/projects
GET /api/projects
GET /api/projects/{id}
PATCH /api/projects/{id}
DELETE /api/projects/{id}

POST /api/projects/{id}/mobs
GET /api/mobs/{mobId}
PATCH /api/mobs/{mobId}

POST /api/mobs/{mobId}/references
POST /api/mobs/{mobId}/ai/analyse
POST /api/mobs/{mobId}/ai/generate-geometry
POST /api/mobs/{mobId}/ai/edit-geometry
POST /api/mobs/{mobId}/ai/generate-texture
POST /api/mobs/{mobId}/ai/edit-texture
POST /api/mobs/{mobId}/ai/generate-animation
POST /api/mobs/{mobId}/ai/edit-animation

POST /api/mobs/{mobId}/validate
POST /api/mobs/{mobId}/export/bbmodel
GET /api/jobs/{jobId}/events
```

Servicios internos

Módulo	Responsabilidad
project-service	CRUD de proyectos, mobs, revisiones y permisos.
asset-service	Referencias, PNG, thumbnails y exports.
ai-orchestrator	Prompts versionados, providers, jobs y schemas.
model-engine	Operaciones geométricas, UV, rig y invariantes.
animation-engine	Presets, tracks, eventos y playback data.
export-service	.bbmodel, companions, ZIP.
validation-service	Blockbench/FMM checks y reportes accionables.

15. Historial, revisiones y seguridad

- Toda propuesta IA aceptada crea una revisión.
- Undo/redo para operaciones manuales y de IA.
- Checkpoints nombrados para estados importantes.
- Autosave local + persistencia de backend.
- No usar eval ni ejecutar código generado por IA.
- Validar todos los JSON estructurados con schema.
- No guardar secretos del proveedor en frontend.
- Los jobs de IA deben ser cancelables y sus fallos no mutar el modelo.

16. Roadmap recomendado

Fase	Entregable vertical
1. Core determinista	Proyecto → mob → cuboides/bones → editar → exportar fixture .bbmodel.
2. IA geometría	Referencia → ModelIntent → GeometryOps → diff → aplicar.
3. Textura	UV packer → pixel editor → live preview → IA por regiones.
4. Animación	AnimationSpec → timeline → presets → IA.
5. FMM polish	Anchor FMM, reporte de compatibilidad, ZIP y fixtures de integración.

Vertical slice inicial

```

Crear proyecto Galgoth
-> Agregar Carcomido
-> cargar/generar modelo cuboidal
-> seleccionar una mano y escalarla
-> exportar .bbmodel
-> abrir en Blockbench
-> probar con FMM

```

17. Criterios de aceptación

1. Crear el proyecto “Galgoth”.
2. Agregar “Carcomido” con la imagen de referencia incluida.
3. Generar una aproximación humanoide editable con cuboides.
4. Seleccionar una mano y cambiar su tamaño manualmente.

5. Pedir a la IA aumentar la otra mano y revisar diff antes de aplicar.
6. Generar o pintar textura y verla en 3D en tiempo real.
7. Enfocar una parte/cara en UV y resaltar su equivalente 3D.
8. Generar una animación Walk pesada/asimétrica.
9. Abrir el timeline avanzado y modificar un keyframe.
10. Validar el modelo sin errores críticos.
11. Exportar .bbmodel + textura/ZIP.
12. Abrir el .bbmodel en Blockbench sin reparaciones manuales.
13. Cargar el modelo bajo el flujo soportado de FreeMinecraftModels y recargarlo correctamente.

18. Prompt maestro para agente de programación

Este paquete incluye MASTER_PROMPT.md. Está escrito para adjuntar toda la carpeta a un agente de programación y comenzar una implementación real. El prompt contiene contratos, rutas, stack, acceptance criteria y el orden recomendado de desarrollo.

Uso recomendado

Adjunta la carpeta completa al agente. Pídele que inspeccione primero mockups/00_all_views.png, references/carcomido_reference.png, samples/ y TECHNICAL_REFERENCES.md. Luego pega MASTER_PROMPT.md íntegro.

19. Referencias técnicas verificadas

Baseline consultado el 2026-09-08. Como .bbmodel y FMM evolucionan, el código debe encapsular sus detalles y probarlos con fixtures.

Fuente	Punto relevante
https://blockbench.net/wiki/docs/bbmodel/	.bbmodel es JSON interno; cambios breaking posibles; v5 separa groups/outliner.
https://blockbench.net/wiki/blockbench/formats/	Capacidades por formato: cubos, bone rotations, animación, UV.
https://github.com/MagmaGuy/FreeMinecraftModels	Importa .bbmodel/.fmmodel, genera resource pack y soporta animaciones.
FMM FileModelConverter.java	Lee meta.format_version, texturas, elements, outliner/groups y animations; maneja diferencias v4/v5.

20. Contenido del build pack

```
galgoth_build_pack/  
  README_START_HERE.md  
  MASTER_PROMPT.md  
  TECHNICAL_REFERENCES.md  
  Galgoth_Studio_Product_AI_Technical_Spec.docx  
  Galgoth_Studio_Product_AI_Technical_Spec.pdf  
  mockups/  
    00_all_views.png  
    01_...png ... 12_...png  
  references/  
    carcomido_reference.png  
    legacy_ui/  
  diagrams/  
    architecture.png  
    ai_pipeline.png  
    workspaces.png  
  samples/  
    carcomido_minecraft_cuboids.bbmodel  
    model_spec_example.json  
    animation_spec_example.json
```

La documentación y los mockups están pensados para que un equipo o un agente de programación pueda construir el editor por incrementos sin perder las decisiones de producto tomadas durante el diseño.